



МИНОБРНАУКИ РОССИИ

федеральное государственное бюджетное образовательное учреждение высшего образования

«Санкт-Петербургский государственный технологический институт (технический университет)»

СПбГТИ(ТУ)

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

**«Разработка информационно-поисковой системы для выбора
теплообменных аппаратов»**

Направление подготовки	09.03.01 — Информатика и вычислительная техника
Направленность	Системы автоматизированного проектирования
Факультет	Информационных технологий и управления
Кафедра	Систем автоматизированного проектирования и управления
Студент	_____
Группа	_____
Руководитель практики	_____

Санкт-Петербург

2026

ЗАДАНИЕ НА ТЕХНОЛОГИЧЕСКУЮ (ПРОЕКТНО-ТЕХНОЛОГИЧЕСКУЮ) ПРАКТИКУ

Параметр	Значение
Тема	Разработка информационно-поисковой системы для выбора теплообменных аппаратов
Студент	_____
Группа	_____
Профильная организация	СПбГТИ(ТУ), Санкт-Петербург
Срок проведения	06.07.2026–19.07.2026
Срок сдачи отчёта	19.07.2026

Календарный план

Задача	Срок
Анализ литературных источников и интернет-ресурсов	1–2 рабочий день
Построение концептуальной и даталогической моделей БД	3–4 рабочий день
Построение UML-диаграмм USER и ADMIN	5 рабочий день
Разработка алгоритма фильтрации и ранжирования	6–7 рабочий день
Тестирование системы и оформление отчёта	8–10 рабочий день

Задание принял к выполнению: _____

Руководитель практики: _____

АННОТАЦИЯ

В отчёте рассмотрена разработка информационно-поисковой системы ThermoSelect для выбора теплообменных аппаратов. Выполнен анализ предметной области, определены информационные потребности пользователя и администратора, сформированы концептуальная и даталогическая модели базы данных. Разработаны алгоритм строгой фильтрации и внутреннего упорядочивания, web-интерфейс, Excel-выгрузка, серверное API и подсистема session-аутентификации.

Каталог включает 50 записей четырёх семейств: пластинчатые, кожухотрубные, воздушные и спиральные теплообменники, в том числе российские модели «Ридан» и аппараты советского типа ЧЗТО. Для каждой записи указаны производитель, модель, гранулярность, область применения, материалы, единый набор общих характеристик, специальные факты и источник.

Приложение реализовано на React 19 и Spring Boot 4.1. Схема данных управляется Flyway; для локального запуска используется файловая H2, для эксплуатации предусмотрена PostgreSQL. Проверка одной командой включает Java integration-тесты, Vitest и production-сборку единого исполняемого JAR.

Ключевые слова: теплообменник, информационно-поисковая система, React, Spring Boot, PostgreSQL, H2, Flyway, CSRF, ранжирование, каталог.

СОДЕРЖАНИЕ

АННОТАЦИЯ	3
СОДЕРЖАНИЕ	4
ВВЕДЕНИЕ.....	6
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	7
1.1 Назначение и типы теплообменных аппаратов	7
1.2 Пользователи и информационные потребности	7
1.3 Источники и качество данных	8
1.4 Состав каталога	9
2 ТРЕБОВАНИЯ И ВАРИАНТЫ ИСПОЛЬЗОВАНИЯ.....	10
2.1 Функциональные требования	10
2.2 Нефункциональные требования	10
2.3 UML вариантов использования	11
2.4 Критерии выбора архитектуры и инструментов	12
2.5 Обоснование web-интерфейса	12
2.6 Выбор frontend-стека	13
2.7 Выбор backend и базы данных.....	14
2.8 Выбор способа сборки и развёртывания	17
3 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ.....	18
3.1 Концептуальная модель	18
3.2 Состав таблиц	18
3.3 Целостность и версии	19
4 АРХИТЕКТУРА И РЕАЛИЗАЦИЯ	20
4.1 Компонентная схема.....	20
4.2 Backend.....	20

4.3 Frontend	20
4.4 Авторизация и безопасность	21
4.5 Профили выполнения	21
5 АЛГОРИТМ ПОИСКА И РАНЖИРОВАНИЯ	22
5.1 Последовательность обработки	22
5.2 Формула внутреннего упорядочивания	22
5.3 Объяснимость	23
6 ТЕСТИРОВАНИЕ	24
6.1 Стратегия	24
6.2 Проверка полноты данных	24
6.3 Ожидаемый результат	24
7 ЗАПУСК И ЭКСПЛУАТАЦИЯ.....	25
7.1 Локальный запуск	25
7.2 Просмотр тестов	25
7.3 Просмотр журналов	25
ЗАКЛЮЧЕНИЕ	26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	27
ПРИЛОЖЕНИЕ А. МАТРИЦА API.....	29
ПРИЛОЖЕНИЕ Б. ОСНОВНЫЕ ТЕСТ-КЕЙСЫ	30
ПРИЛОЖЕНИЕ В. СТРУКТУРА ПРОЕКТА	31

ВВЕДЕНИЕ

Теплообменные аппараты применяются в отоплении, вентиляции, холодильной технике, энергетике, пищевой и химической промышленности. Производители публикуют сведения в разных форматах: web-карточках, листах конкретной конфигурации и каталогах серий. Одинаковые по назначению аппараты могут отличаться конструкцией, материалами, допустимыми температурами, давлением, диапазонами расхода и требованиями к обслуживанию.

Ручной просмотр нескольких каталогов затрудняет первичный выбор. Дополнительную проблему создаёт различная гранулярность сведений: значение для конкретного артикула можно использовать непосредственно, а диапазон серии требует уточнения комплектации. Мощность теплообменника также не является постоянной паспортной величиной и зависит от рабочих сред, расходов, температурного графика и допустимого гидравлического сопротивления.

Цель практики — разработать информационно-поисковую web-систему, которая хранит структурированные сведения о теплообменниках, выполняет строгую фильтрацию, ранжирует результаты и объясняет причины совпадения без подмены инженерного теплового расчёта.

Для достижения цели решены следующие задачи:

- проанализированы официальные материалы производителей;
- разработаны концептуальная, логическая и физическая модели данных;
- определены роли пользователя и администратора, построены UML-сценарии;
- реализованы регистрация, вход, каталог, фильтры, карточка, сравнение и административные функции;
- разработан алгоритм нормализованного ранжирования;
- подготовлены тесты, инструкции по запуску, журналы и материалы защиты.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Назначение и типы теплообменных аппаратов

Теплообменник обеспечивает передачу теплоты между средами, разделёнными поверхностью теплообмена либо контактирующими в специально организованном процессе. В границах проекта рассматриваются четыре семейства аппаратов.

Семейство	Конструктивная особенность	Типовые области применения
Пластинчатые	Пакет профилированных пластин; разборное или паяное исполнение	отопление, охлаждение, HVAC, холодильная техника
Кожухотрубные	Пучок труб внутри кожуха; возможны разные материалы и число ходов	промышленность, энергетика, масла, морские системы
Воздушные	Оребренный теплообменный блок с вентиляторами	конденсация, drycooling, холодильные установки, дата-центры
Спиральные	Один или два спиральных канала, устойчивых к загрязнению	загрязнённые среды, рекуперация, конденсация, сточные воды

Система не выполняет окончательный подбор. Она сокращает множество кандидатов и формирует объяснимый список для последующего обращения к производителю или выполнения инженерного расчёта.

1.2 Пользователи и информационные потребности

Обычному пользователю необходимы регистрация, вход, просмотр справочников, поиск по типу и характеристикам, выгрузка найденных вариантов в Excel, детальная карточка и сравнение нескольких вариантов. Интерфейс показывает значимые характеристики и гранулярность записи, не перегружая результат служебным баллом или процентом совпадения.

Администратор поддерживает качество каталога: создаёт и редактирует записи, публикует или архивирует их, управляет производителями и пользователями. Система запрещает администратору удалять, блокировать или лишать роли собственную учётную запись, чтобы не потерять управление.

1.3 Источники и качество данных

Исходные сведения собраны с ресурсов Alfa Laval, Danfoss, Kelvion, SWEP, API Heat Transfer, «Ридан» и каталога советского типоряда ЧЗТО. Каждая запись содержит URL, название источника, дату проверки и текстовое основание измерений. Дополнительно хранится гранулярность.

Гранулярность	Смысл	Уровень доверия при ранжировании
EXACT_CONFIGURATION	конкретный SKU или заранее определённая комплектация	высокий
STANDARD_MODEL	стандартная модель с опубликованными параметрами	средний
SERIES	конфигурируемая серия и её общий диапазон	требует уточнения

Открытые каталоги не содержат одинаковый набор характеристик для всех 50 записей. Поэтому общая часть карточки нормализована: площадь, диапазоны расхода и температуры, максимальное давление, габариты и масса. При отсутствии отдельного значения в открытом материале используется согласованное справочное значение для соответствующего семейства и типоразмера. Мощность исключена из общего набора, так как без рабочего режима она не является сопоставимой характеристикой модели.

1.4 Состав каталога

Производитель	Семейство	Количество
Alfa Laval	пластинчатые Fast Track	8
Danfoss	пластинчатые B3	6
Kelvion	пластинчатые NX	6
SWEP	пластинчатые All-Stainless	4
Ридан	пластинчатые HH	4
Basco	кожухотрубные	6
ЧЗТО	кожухотрубные советского типоряда	4
Kelvion	воздушные	6
Alfa Laval	спиральные SHE	6
Итого		50

2 ТРЕБОВАНИЯ И ВАРИАНТЫ ИСПОЛЬЗОВАНИЯ

2.1 Функциональные требования

Публичная часть содержит лендинг, регистрацию и вход. После авторизации пользователь получает доступ к каталогу, фильтрам, Excel-выгрузке, карточкам, сравнению и read-only странице учётной записи. Административная часть содержит обзор, список записей, редактор, смену статусов и управление пользователями.

Строгие числовые фильтры должны исключать аппарат, который не покрывает требование. Пагинация должна быть стабильной. Сравнение принимает от двух до четырёх уникальных идентификаторов и сохраняет порядок пользователя. Удаление записи каталога заменяется архивированием.

2.2 Нефункциональные требования

- единый исполняемый JAR и один origin для UI/API;
- русский адаптивный интерфейс;
- относительные API URL и отсутствие production CORS;
- Flyway как единственный владелец схемы;
- optimistic locking для административных изменений;
- единый JSON-формат ошибок;
- 30-минутная session-аутентификация и немедленный отзыв доступа;
- воспроизводимые Java- и React-тесты.

2.3 UML вариантов использования

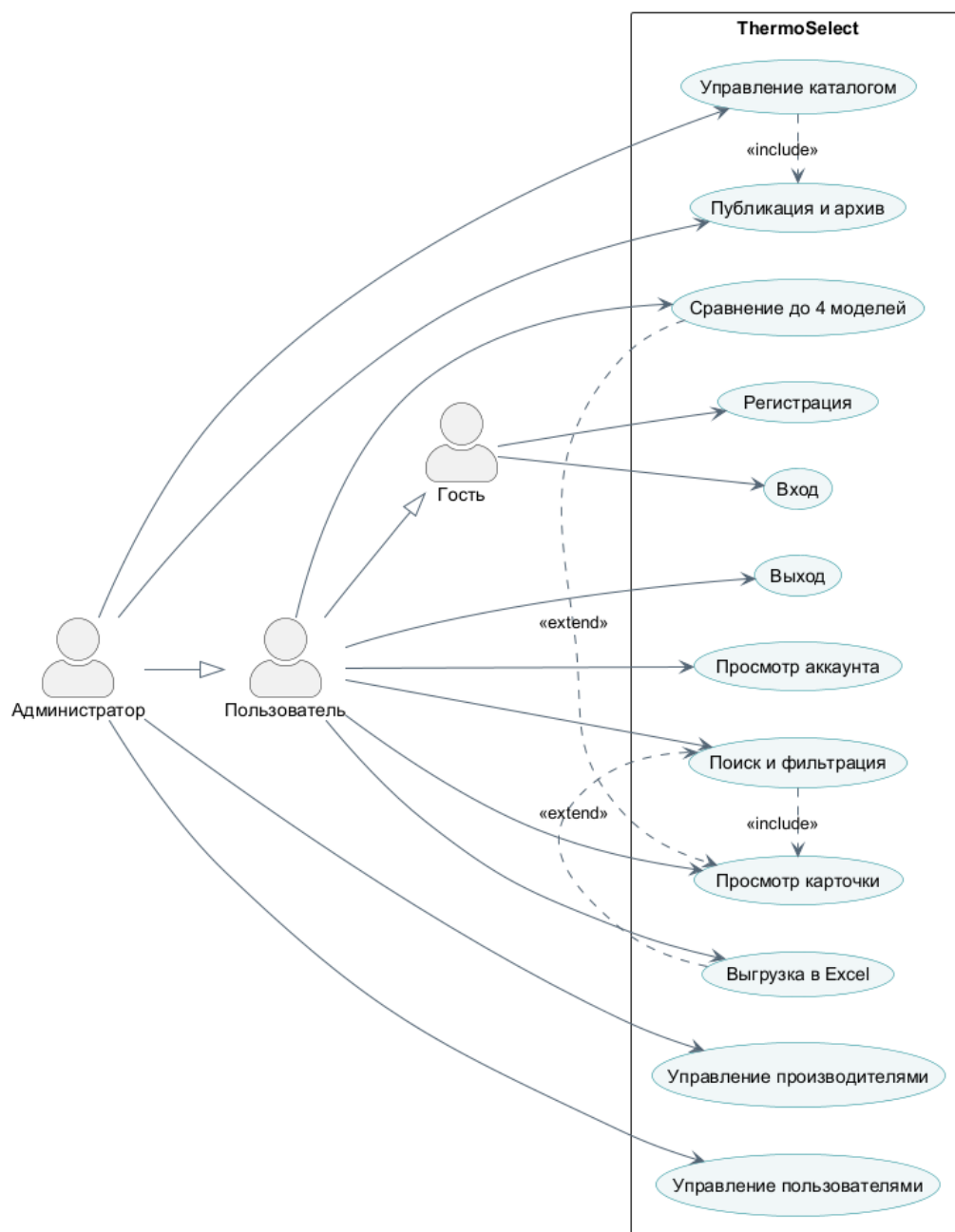


Рисунок 1 – Варианты использования ThermoSelect

Пользователь работает только с опубликованными данными. Администратор наследует пользовательские сценарии и получает команды изменения. Проверка роли выполняется сервером для каждого ADMIN endpoint; скрывание пункта меню во frontend не является механизмом безопасности.

2.4 Критерии выбора архитектуры и инструментов

Технологии выбирались не по популярности как самостоятельному показателю, а по соответствию ограничениям проекта. Основными критериями стали: реализация всех пользовательских и административных сценариев, безопасность session-аутентификации, строгая работа с реляционными данными, возможность запуска на разных компьютерах, воспроизводимая сборка, автоматическое тестирование и завершение работы в срок учебной практики.

При сравнении учитывались также эксплуатационные свойства. Решение должно запускаться без установки отдельного frontend-сервера, работать с одной точкой входа, не требовать CORS в production и сохранять возможность перехода с локальной базы на серверную СУБД. Поэтому «лучшим» далее называется вариант, который получил наилучшее соответствие именно этим требованиям. При других условиях — например, обязательной автономной работе без браузера или наличии корпоративного стандарта .NET — итоговый выбор мог бы отличаться.

2.5 Обоснование web-интерфейса

Для пользовательского интерфейса сравнивались web-приложение, настольное приложение JavaFX, desktop-оболочка Electron и консольная программа.

Форма	Преимущества	Ограничения для ThermoSelect	Выбор
Web SPA	запуск в браузере, единое обновление, адаптивность, доступ с разных ОС, удобные формы и таблицы	требуется браузер и запущенный сервер	выбран
JavaFX	полноценное desktop-приложение, доступ к возможностям ОС, возможна автономная работа	отдельная установка и доставка обновлений, дополнительная реализация адаптивного интерфейса	не выбран
Electron	общий стек HTML, CSS и JavaScript, кроссплатформенная desktop-сборка	в поставку включаются Chromium и Node.js; отдельный установщик не решает задачу централизованного каталога	не выбран

Форма	Преимущества	Ограничения для ThermoSelect	Выбор
CLI	минимальные требования к интерфейсу и ресурсам	неудобны фильтры, сравнение, графическое объяснение рейтинга и администрирование	не выбран

Web-интерфейс выбран как наиболее подходящий, поскольку каталог является многопользовательской информационной системой, а не локальным инженерным калькулятором. Браузер уже предоставляет стандартные элементы форм, навигацию, доступность и адаптацию к экрану. Размещение React и API на одном origin упрощает cookie-сессии и CSRF-защиту. Установка клиента на каждое рабочее место не требуется: для обновления достаточно заменить серверный JAR. Electron был бы оправдан при необходимости автономной desktop-работы или интеграции с локальным оборудованием, которых в задании нет.

2.6 Выбор frontend-стека

React сопоставлялся с Vue и Angular по шкале от 1 до 5. Вес показывает значимость критерия для текущего проекта; итоговая оценка является взвешенной суммой и служит прозрачным обоснованием, а не универсальным рейтингом технологий.

Критерий	Вес, %	React	Vue	Angular
Компонентная реализация сложных экранов	25	5	5	5
Совместимость с JavaScript и JSDoc	20	5	5	2
Скорость освоения и реализации	15	4	5	2
Тестирование и маршрутизация	15	5	4	5
Контроль состава зависимостей	10	4	4	2
Зрелость экосистемы и доступность компонентов	15	5	4	5
Взвешенный итог	100	4,75	4,60	3,65

React предоставляет компонентную модель и не навязывает полный набор подсистем. Это удобно для проекта, где маршрутизация выбирается

отдельно, а глобальное состояние ограничено контекстами авторизации и сравнения. Vue является сильным аналогом: он также компонентный, декларативный и отличается низким порогом входа. React получил преимущество за более широкую экосистему библиотек и учебных материалов, а также за удобное сочетание JavaScript с JSDoc-контрактами без обязательного перехода на TypeScript. Angular предоставляет целостный framework, строгую структуру и развитые средства для крупных команд, но предполагает более тяжёлую модель приложения и освоение TypeScript, декораторов, RxJS и дополнительных концепций. Для ограниченного по срокам проекта это увеличивает стоимость реализации без добавления требуемых предметных функций.

Vite выбран вместо ручной конфигурации Webpack как готовая цепочка разработки и production-сборки с быстрым dev-сервером, HMR и оптимизированными статическими ресурсами. Vitest использует совместимую с Vite среду и не требует второй независимой конфигурации преобразования JSX. Redux не применён, поскольку разделяемое состояние невелико: добавление отдельного хранилища увеличило бы число абстракций без измеримого упрощения текущих сценариев. UI-фреймворк также не используется, чтобы сохранить инженерный визуальный стиль и не включать большой набор невостребованных компонентов.

2.7 Выбор backend и базы данных

Backend-стек выбирался с нуля между Spring Boot, NestJS и ASP.NET Core. Все три варианта позволяют построить REST API, применить dependency injection, выполнить валидацию и покрыть приложение автоматическими тестами. Для сравнения использована шкала от 1 до 5. Наибольший вес получили безопасность и транзакционная работа с реляционными данными, а требование единого исполняемого JAR рассматривалось как обязательное условие поставки.

Критерий	Вес, %	Spring Boot	NestJS	ASP.NET Core
----------	--------	-------------	--------	--------------

Критерий	Вес, %	Spring Boot	NestJS	ASP.NET Core
Session-аутентификация, CSRF и роли	25	5	3	5
Транзакции и реляционная модель	20	5	4	5
Сборка в исполняемый JAR	20	5	1	1
Статическая типизация предметной модели	15	5	4	5
Интеграционное тестирование	10	5	4	5
Простота воспроизводимой сборки	10	4	4	4
Взвешенный итог	100	4,90	3,15	4,10

NestJS привлекателен единым языком TypeScript на frontend и backend, модульной архитектурой, guards и удобным быстрым стартом. Он хорошо подходит для I/O-нагруженных API и позволяет выбирать Express либо Fastify. Однако для требуемой session-аутентификации необходимо согласовать несколько отдельных механизмов и middleware, а итоговая поставка зависит от Node.js и не соответствует формату JAR. Для данного проекта преимущество общего языка не компенсирует менее прямое выполнение требований безопасности и упаковки.

ASP.NET Core является наиболее близким конкурентом Spring Boot. Он предлагает строгую типизацию C#, встроенный dependency injection, развитую модель middleware, средства авторизации, тестирования и высокопроизводительный сервер Kestrel. При выборе экосистемы .NET это было бы полноценное production-решение. Его итоговая оценка ниже только из-за целевого формата поставки: приложение должно собираться средствами Maven и запускаться как Java JAR. Переход к .NET изменил бы модель сборки и формат артефакта, не добавляя необходимых предметных функций.

Spring Boot получил наибольшую оценку как наиболее цельное решение для заданных условий. Spring Security объединяет session-аутентификацию, CSRF, разграничение ролей и обработку отказов в одном

фильтровом конвейере. Spring MVC и Bean Validation обеспечивают типизированные входные контракты и единообразную проверку запросов. Spring Data JPA и декларативные транзакции согласуются с нормализованной моделью каталога, optimistic locking и PostgreSQL. Spring Boot Test, JUnit и MockMvc позволяют проверять API вместе с контекстом безопасности и базой данных. Наконец, Maven фиксирует версии зависимостей, а Spring Boot Maven Plugin создаёт единый исполняемый JAR с сервером и frontend-ресурсами. Таким образом, Spring Boot выбран по совокупности преимуществ в безопасности, целостности данных, тестируемости и простоте поставки.

Для production-профиля сравнивались PostgreSQL, MySQL и SQLite. PostgreSQL и MySQL поддерживают транзакции, ограничения и многопользовательскую работу. PostgreSQL выбран благодаря строгой реляционной модели, развитым ограничениям целостности, MVCC и поддержке стандартных уровней изоляции. Эти свойства соответствуют административному CRUD и optimistic locking. MySQL остаётся допустимой альтернативой, но потребовал бы отдельной проверки SQL-совместимости миграций. SQLite удобен как встраиваемый файл, однако не является целевой серверной СУБД для одновременной работы пользователей и администраторов.

H2 используется только для автоматических тестов и локального запуска: она не требует установки отдельной СУБД и сохраняет данные между запусками. Такое разделение сочетает простой запуск на учебном компьютере и корректную production-архитектуру. Flyway выбран единственным владельцем схемы, потому что версионизируемые миграции воспроизводимы и применимы как к чистой, так и к существующей базе; Hibernate настроен на проверку схемы, а не на её неявное изменение.

2.8 Выбор способа сборки и развёртывания

Рассматривались два варианта: независимые frontend и backend с отдельными серверами либо модульный монолит в одном JAR. Раздельное развёртывание полезно, когда части масштабируются независимо, имеют разные команды сопровождения или требуют CDN. Для ThermoSelect эти преимущества не компенсируют настройку двух поставок, CORS, отдельных адресов и координацию версий API.

Выбран единый исполняемый JAR. Maven устанавливает закреплённый Node, выполняет `npm ci`, React-тесты и `Vite build`, копирует статические файлы и только затем упаковывает Spring Boot. В результате одной командой проверяются обе части, а демонстрация требует один процесс. Микросервисное разбиение также отклонено: авторизация, каталог и администрирование используют одну транзакционную модель и не имеют независимой нагрузки, поэтому распределённые сессии, сетевые отказы и несколько баз данных создали бы сложность без пользы для задания.

3 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ

3.1 Концептуальная модель

Центральная сущность `heat_exchangers` связана с производителем, областями применения, материалами, типоспецифичными фактами, источниками и температурно-барическими ограничениями. Пользователи хранятся отдельно, поскольку их жизненный цикл не зависит от каталога.

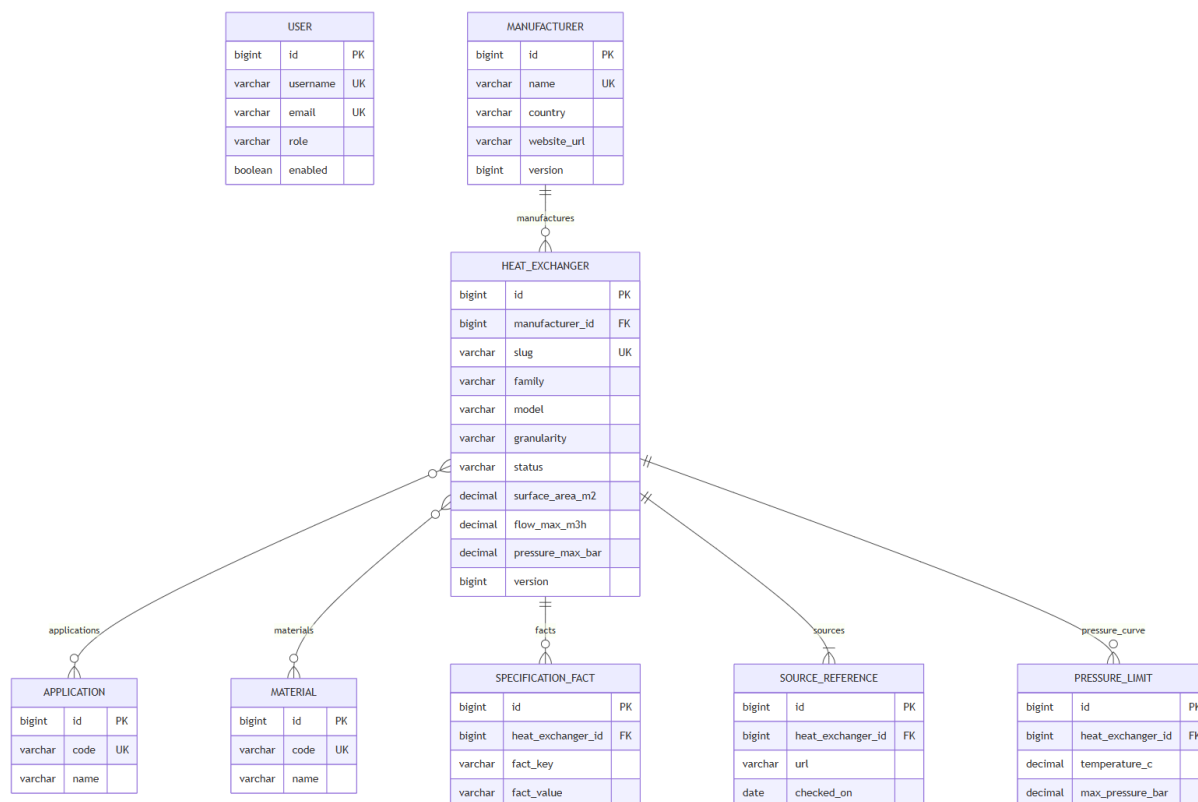


Рисунок 2 – Концептуальная и даталогическая модель

Модель разделяет общие фильтруемые атрибуты и специальные факты. Это позволяет эффективно выполнять числовые ограничения и не расширять таблицу отдельным столбцом для каждой конструктивной особенности.

3.2 Состав таблиц

Таблица	Назначение	Основные поля
users	учётные записи и роли	username, email, password_hash, role, enabled, version
manufacturers	производители	name, country, website_url, version

Таблица	Назначение	Основные поля
heat_exchangers	общие параметры карточки	slug, family, model, granularity, status, площадь, расход, температура, максимальное давление, габариты, масса, version
applications	справочник применений	code, name
materials	справочник материалов	code, name
heat_exchanger_applications	М:N аппаратов и применений	heat_exchanger_id, application_id
heat_exchanger_materials	М:N аппаратов и материалов	heat_exchanger_id, material_id
specification_facts	специальные свойства	fact_key, label, value, unit, sort_order
source_references	источники	title, url, checked_on, measurement_basis
pressure_limits	точки температурно-барической кривой	temperature_c, max_pressure_bar, note

3.3 Целостность и версии

Уникальные ограничения установлены для имени производителя, кодов справочников, username, email и slug. Внешние ключи предотвращают появление фактов или источников без родительской записи. Поле version используется JPA для optimistic locking: если два администратора открыли одну форму, сохранение устаревшей версии завершается конфликтом HTTP 409.

Статусы DRAFT, PUBLISHED и ARCHIVED отделяют подготовку данных от пользовательского каталога. Физическое удаление аппарата не выполняется: DELETE переводит запись в ARCHIVED.

4 АРХИТЕКТУРА И РЕАЛИЗАЦИЯ

4.1 Компонентная схема

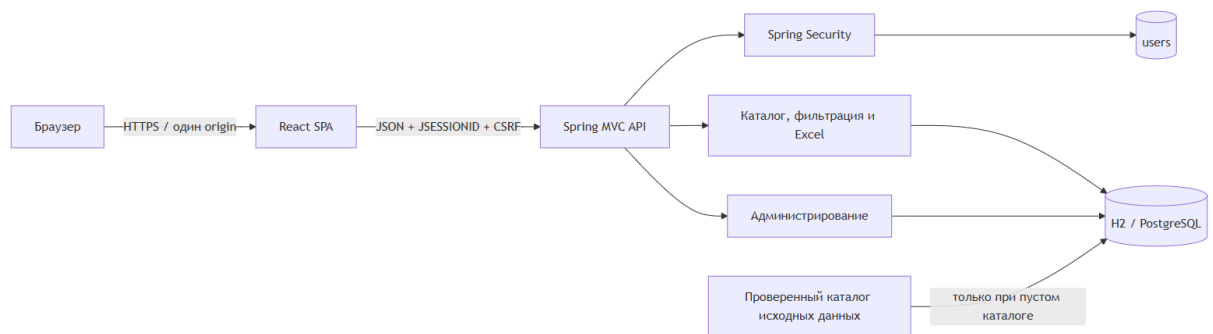


Рисунок 3 – Компоненты приложения

React-приложение собирается Vite в статические ресурсы Spring Boot. В production браузер обращается к UI и API с одного origin. Maven устанавливает закреплённый Node, выполняет `npm ci`, тесты и сборку frontend, затем Spring Boot Maven Plugin формирует исполняемый JAR.

4.2 Backend

Backend разделён на контроллеры, сервисы, репозитории и JPA-модель. Пользовательские endpoint не возвращают DRAFT и ARCHIVED. Административный DTO требует текущую version, валидные справочные коды и хотя бы один источник.

Единый ответ ошибки имеет поля timestamp, status, error, code, message, details и path. Предметные коды позволяют frontend различать ошибки валидации, CSRF, отсутствие сущности и конфликт версии.

4.3 Frontend

Frontend написан на JavaScript с JSDoc-контрактами. React Router разделяет публичные, пользовательские и административные маршруты. AuthContext загружает CSRF и текущего пользователя, CompareContext хранит до четырёх идентификаторов. Redux и UI-фреймворк не используются.

Каталог синхронизирует фильтры с URL. На мобильном экране фильтры превращаются в выдвижную панель, а сравнение допускает горизонтальную прокрутку. Каталог, карточка и сравнение используют единый формат чисел: не более одного знака после запятой, а размеры в миллиметрах округляются до целых. Служебные проценты и технические метки заполнения пользователю не выводятся.

4.4 Авторизация и безопасность

Пароли хранятся как BCrypt-хеши. До изменяющего запроса клиент получает CSRF-токен. При JSON-входе вызывается `ChangeSessionIdAuthenticationStrategy`, `SecurityContext` явно сохраняется в `HttpSession`, а использованный токен инвалидируется.

Cookie сессии имеет `HttpOnly` и `SameSite=Lax`. В production профиль PostgreSQL по умолчанию требует secure cookie. Фильтр защищённых запросов перечитывает пользователя из БД, поэтому блокировка, удаление или смена роли действуют без ожидания истечения старой сессии.

4.5 Профили выполнения

Профиль	База	Назначение
test	H2 in-memory в режиме PostgreSQL	автоматические тесты
demo	файловая H2	локальный запуск, данные сохраняются
postgres	внешняя PostgreSQL	эксплуатационный запуск

5.3 Объяснимость

Для диагностики API может формировать текстовые причины: соответствие модели или производителя, область применения, материал, достаточный запас параметра и гранулярность записи. Пользовательский интерфейс оставляет только полезные паспортные сведения и не показывает служебные проценты или технические метки заполнения.

6 ТЕСТИРОВАНИЕ

6.1 Стратегия

Java integration-тесты поднимают Spring context, применяют Flyway и проверяют репозитории, сервисы, HTTP и Spring Security. React-тесты используют Vitest и Testing Library. Полная команда `mvnw.cmd clean verify` не требует отдельно установленного Maven или Node.

Группа	Проверяемые сценарии
Auth	CSRF, регистрация, duplicate email, вход, session fixation, роли, отзыв сессии
Catalog	50 записей, полнота полей, фильтры, пагинация, сравнение, Excel-выгрузка
Admin	CRUD, публикация, архивирование, optimistic lock
Frontend	AuthContext, параметры фильтра, лимит сравнения
Packaging	npm build, static resources, исполняемый JAR

6.2 Проверка полноты данных

Автоматический тест требует, чтобы у каждой из 50 записей были заполнены площадь, минимальный и максимальный расход, температура, максимальное давление, ширина, высота, глубина, масса и хотя бы две точки температурно-барической кривой. Также проверяются URL и дата источника, наличие российских моделей и возможность открыть сформированную Excel-книгу.

6.3 Ожидаемый результат

Успешная сборка заканчивается `BUILD SUCCESS`. XML и текстовые результаты Java находятся в `target/surefire-reports`, frontend-вывод — в журнале Maven. Итоговый JAR создаётся в `Авторизация/target/`.

7 ЗАПУСК И ЭКСПЛУАТАЦИЯ

7.1 Локальный запуск

```
cd .\Авторизация  
.\run-demo.ps1
```

После сборки интерфейс доступен на `http://localhost:8080`.
Локальные учётные записи: `demo/demo12345` и `admin/admin12345`. Скрипт совместим с Windows PowerShell 5.1 и PowerShell 7.

7.2 Просмотр тестов

```
cd .\Авторизация  
.\mvnw.cmd clean verify  
Get-ChildItem .\target\surefire-reports  
Get-Content .\target\surefire-reports\*.txt
```

7.3 Просмотр журналов

```
Get-Content .\Авторизация\logs\thermoselect.log -Tail 100  
Get-Content .\Авторизация\logs\thermoselect.log -Tail 50 -Wait
```

Журнал ротируется, хранится семь дней и не включается в Git. Отчёт по тестам хранится отдельно от runtime-журнала.

ЗАКЛЮЧЕНИЕ

В ходе учебной практики разработана full-stack система ThermoSelect, соответствующая индивидуальному заданию. Проведён анализ предметной области и официальных каталогов, построены модели БД и UML, реализованы алгоритм поиска, пользовательский интерфейс, серверное API, защита сессий и административные функции.

Каталог объединяет 50 записей разных конструкций, включая российские модели и советские типоразмеры. Общие характеристики унифицированы, числовые значения показываются с разумной точностью, а найденные аппараты можно выгрузить в Excel. Проект собирается одной командой в исполняемый JAR и сопровождается тестами, журналами, инструкциями и материалами защиты.

Дальнейшее развитие может включать полноценный тепловой расчёт с заданными средами и температурным графиком, импорт паспортов, историю поисков и избранное. Эти функции требуют отдельной валидации и не входят в текущую информационно-поисковую версию.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Spring Boot Reference Documentation. URL: <https://docs.spring.io/spring-boot/4.1-SNAPSHOT/reference/> (дата обращения: 13.07.2026).
2. Spring Security CSRF. URL: <https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html> (дата обращения: 13.07.2026).
3. React Documentation. URL: <https://react.dev/> (дата обращения: 13.07.2026).
4. Vite Build Guide. URL: <https://vite.dev/guide/build> (дата обращения: 13.07.2026).
5. Alfa Laval Fast Track. URL: <https://shop.alfalaval.com/en-us/gasketed-plate-heat-exchangers--2244334> (дата обращения: 13.07.2026).
6. Danfoss BPHE B3-012, 111B6315. URL: <https://designcenter.danfoss.com/products/climate-solutions-for-cooling/heat-exchangers/brazed-plate-heat-exchangers/fishbone-brazed-plate-heat-exchangers/bphe-b3/p/111B6315> (дата обращения: 13.07.2026).
7. Kelvion Gasketed Plate Heat Exchangers. URL: <https://www.kelvion.com/products/plate-heat-exchangers/gasketed-plate-heat-exchangers/> (дата обращения: 13.07.2026).
8. SWEP All-Stainless. URL: <https://www.swepgroup.com/challenge-efficiency/technology/swep-all-stainless> (дата обращения: 13.07.2026).
9. API Heat Transfer Basco Type 500. URL: <https://www.apiheattransfer.com/product/type-500/> (дата обращения: 13.07.2026).
10. Kelvion Flatbed Condenser / Drycooler. URL: <https://www.kelvion.com/products/heat-rejection-heat-recovery-solutions/flatbed-condensers/-drycooler> (дата обращения: 13.07.2026).
11. Alfa Laval SHE LTL product leaflet. URL: <https://assets.alfalaval.com/documents/pc168354a/alfa-laval-product-leaflet-she-ltl-en.pdf> (дата обращения: 13.07.2026).

12. Alfa Laval SHE Cond product leaflet. URL: <https://assets.alfalaval.com/documents/p36beedba/alfa-laval-product-leaflet-she-cond-en.pdf> (дата обращения: 13.07.2026).
13. Ридан. Каталог разборных пластинчатых теплообменников НН. URL: <https://oldteploobmennik.ridan.ru/products/catalog-rpto/> (дата обращения: 16.07.2026).
14. Каталог кожухотрубных теплообменников, холодильников и испарителей советского типоряда. URL: <https://ohladiteli.nt-rt.ru/images/manuals/teploobmennik.pdf> (дата обращения: 16.07.2026).
15. Angular Documentation. What is Angular? URL: <https://angular.dev/overview> (дата обращения: 13.07.2026).
16. Vue.js Guide. Introduction. URL: <https://vuejs.org/guide/introduction> (дата обращения: 13.07.2026).
17. NestJS Documentation. First steps. URL: <https://docs.nestjs.com/first-steps> (дата обращения: 13.07.2026).
18. Microsoft Learn. Overview of ASP.NET Core. URL: <https://learn.microsoft.com/en-us/aspnet/core/overview> (дата обращения: 13.07.2026).
19. PostgreSQL. About. URL: <https://www.postgresql.org/about/> (дата обращения: 13.07.2026).
20. MySQL 8.4 Reference Manual. Transactional and Locking Statements. URL: <https://dev.mysql.com/doc/en-us/mysql-8.4-en/innodb-transaction-model.html> (дата обращения: 13.07.2026).
21. Electron Documentation. Introduction. URL: <https://www.electronjs.org/docs/latest/> (дата обращения: 13.07.2026).
22. Vite Guide. Why Vite. URL: <https://vite.dev/guide/why> (дата обращения: 13.07.2026).
23. Spring Boot. Launching Executable Jars. URL: <https://docs.spring.io/spring-boot/specification/executable-jar/launching.html> (дата обращения: 13.07.2026).

ПРИЛОЖЕНИЕ А. МАТРИЦА API

Доступ	Endpoint	Назначение
Public	GET /api/auth/csrf	получение CSRF
Public	POST /api/auth/register, /login	регистрация и вход
USER/ADMIN	GET /api/auth/me, POST /logout	профиль и выход
USER/ADMIN	POST /api/heat-exchangers/search	поиск и упорядочивание
USER/ADMIN	POST /api/heat-exchangers/export	Excel-выгрузка текущего фильтра
USER/ADMIN	GET /api/heat-exchangers/{slug}	карточка
USER/ADMIN	POST /api/heat-exchangers/compare	сравнение 2–4 позиций
ADMIN	/api/admin/heat-exchangers/**	CRUD и статусы
ADMIN	/api/admin/manufacturers/**	производители
ADMIN	/api/admin/users/**	роли, блокировка, удаление

ПРИЛОЖЕНИЕ Б. ОСНОВНЫЕ ТЕСТ-КЕЙСЫ

ID	Сценарий	Ожидаемый результат
AUTH-01	Регистрация с CSRF	201, роль USER, BCrypt-хеш
AUTH-02	Повторный email	409 DUPLICATE_USER
AUTH-03	Вход	session ID изменён, /me доступен
AUTH-04	POST без CSRF	403 CSRF_MISSING
AUTH-05	Блокировка активного пользователя	следующий запрос отклонён
CAT-01	Поиск без фильтров	50 опубликованных записей
CAT-02	Числовые требования	каждый результат покрывает ограничение
CAT-03	Полнота каталога	все общие поля заполнены
CAT-04	Сравнение четырёх	порядок сохранён, значения отображены
CAT-05	Экспорт отфильтрованной выдачи	валидная Excel-книга с теми же записями
ADM-01	Обновление с текущей version	200, version увеличена
ADM-02	Повтор устаревшего обновления	409 OPTIMISTIC_LOCK_CONFLICT
ADM-03	Удаление аппарата	статус ARCHIVED, физическая запись сохранена
UI-01	Пятый элемент сравнения	отклонён с сообщением
UI-02	Прямой SPA URL	сервер возвращает React index

ПРИЛОЖЕНИЕ В. СТРУКТУРА ПРОЕКТА

```
Авторизация/  
  frontend/                                React, маршруты, компоненты и Vitest  
  src/main/java/.../catalog Backend каталога  
  src/main/java/.../security Авторизация и ADMIN users  
  src/main/resources/                      Flyway, профили, 50 записей  
  src/test/                                Java integration-тесты  
docs/  
  frontend/ backend/ auth/ tests/ logs/ reports/ defense/  
tools/  
  catalog/                                обогащение и аудит данных  
  reports/                                генерация DOCX
```